

Appendix

A JSON-RPC interface details

The standard auto-generates a JSON-RPC 2.0 server from any `Benchmark` subclass. The server exposes two endpoints whose names and payload shape match MCP’s `tools/list` and `tools/call`, so any MCP-compatible client (LLM tool-call loop, IDE plugin, evaluation harness) can drive a CUBE task without an adapter layer.

Endpoint: `tools/list`. Returns the action schema derived from the task’s default tool: one entry per `@tool_action`-decorated method, with name, description, and JSON Schema for the arguments. The schema is generated at task construction time and is static for the lifetime of the episode.

Endpoint: `tools/call`. Accepts `{name, arguments}` and routes to `task.step(Action(name, arguments))`, returning the serialized `EnvironmentOutput` (observation, done flag, reward if terminal, info dict). A call after `done=True` raises a protocol error; the client must call `tools/list` on a fresh session to start a new episode.

Server lifecycle. `Benchmark._setup()` starts the server (one per benchmark, shared across tasks); `Benchmark.close()` tears it down. Individual tasks register and deregister their endpoints on `reset()` and episode end. The server address is published in `RuntimeContext` so remote workers can connect without further coordination.

B Per-benchmark task count, license, infra requirements

Table 2: The CUBE registry at submission.

CUBE	Modality	# Tasks	Backend
WorkArena	Web	333	Live enterprise web app
WebArena-Verified	Web	812	Shared 6-service Docker stack
MiniWoB	Web	125	Local browser
BrowseComp	Web	1,266	Browser
OSWorld	CUA	368	Ubuntu 22.04 VM
Windows Agent Arena	CUA	154	Windows 11 VM
SWE-bench Verified	SWE	500	Per-task Docker
SWE-bench Live	SWE	1,895	Per-task Docker
Terminal-Bench	Terminal	89	Per-task Docker
DRBench	Research	100	Per-task Docker (Nextcloud, Mattermost, IMAP)
Total		5,642	

C Judge output schema and blame taxonomy

Output schema. Each `JudgeOutput` carries five fields: an *outcome* (success, success_lucky, almost, failure, should_have_been_rewarded); a *primary blame* from the closed taxonomy plus optional *secondary blames*; *evidence*, the step-indexed verbatim quotes; a *hypothesis* of what would fix the failure class; and 0–5 confidence scores on both blame and hypothesis. A free-form *analysis* field placed first acts as a reasoning scratchpad before the structured fields are committed.

Blame taxonomy. Ten benchmark-agnostic categories grouped into three buckets. **Agent-side:** `model_capability` (the LLM lacked the reasoning, knowledge, or planning to solve the task); `agent_scaffolding` (the agent loop, prompt, or context management caused the failure); `submission_format` (the agent reached a solution but failed to submit it through the registered channel). **Tool-side:** `tool_failure` (a tool wrapper raised or returned an unexpected error); `action_space_limited` (no available action could express the correct solution); `insufficient_observation` (the rendered observation hid information the agent needed). **Benchmark-side:** `env_failure` (infrastructure crash outside the agent’s and tool’s control);

task_unclear (the task description is ambiguous or contradictory); eval_brittle (the evaluator rejected an acceptable solution). A residual none category is assigned on clean success or when evidence is insufficient for any attribution.

D Drift sources in dynamic-agent benchmarks

Five sources of drift recur across the corpus and motivate the reliability framing of Appendix D.

Scaffolding sensitivity. Pass rate depends heavily on the agent loop, the tool wrappers, the prompt boilerplate, and the exact context-management strategy. Two scaffolds calling the same model can differ by tens of points on the same benchmark; cross-benchmark comparisons that mix scaffolds are not measuring the model.

Prompt tailoring. Many published numbers use a prompt or chain-of-thought structure tuned for a specific benchmark, sometimes with in-context examples drawn from its training split. The reported number is then a joint property of the model and the prompt-engineering effort, not of the model in a generalist setting.

Model version drift. Frontier APIs are continuously retrained and behaviour-tuned. A model identifier (claude-sonnet-4-6, gpt-5.4) names a moving target whose behaviour at the time of a published number may not be reproducible months later.

Closed-source scaffolds. Strong leaderboard numbers are sometimes reported by labs whose scaffold, tool stack, and prompt are not released, in contexts where headline numbers carry commercial weight. These results are not reproducible by construction.

Live-component drift. Several benchmarks in the corpus depend on live external services or system state that change without notice: WorkArena targets a live enterprise web application; OSWorld and Windows Agent Arena depend on OS revisions; underlying browser-automation libraries (playwright, pyautogui) ship breaking updates between minor versions. Two runs weeks apart against the same task set can produce different ground-truth pass rates.

E Judge meta-analysis

For every episode that received three independent judge invocations we measure *primary-blame agreement* (do all three judges choose the same blame category?) and *outcome agreement* (do all three agree on pass / almost / fail?). Tables 3 and 4 report the counts and rates for the two benchmarks where triple-judge runs were completed: SWE-bench Verified (Sonnet 4.6, $n=162$; GPT-5.4, $n=204$) and SWE-bench Live (Sonnet 4.6, $n=83$).

Table 3: Primary-blame agreement across three independent judges. 3/3: all three choose the same category. 2/3: a two-judge majority. All differ: no two judges agree. $\geq 2/3$ subsumes both 3/3 and 2/3 rows.

Benchmark	Model	n	3/3	2/3	All differ	3/3 %	$\geq 2/3$ %
SWE-bench Verified	Sonnet 4.6	162	117	40	5	72.2	96.9
SWE-bench Verified	GPT-5.4	204	173	27	4	84.8	98.0
SWE-bench Live	Sonnet 4.6	83	64	19	0	77.1	100.0

Table 4: Outcome agreement (pass / almost / fail) across three independent judges. Column definitions identical to Table 3.

Benchmark	Model	n	3/3	2/3	All differ	3/3 %	$\geq 2/3$ %
SWE-bench Verified	Sonnet 4.6	162	114	46	2	70.4	98.8
SWE-bench Verified	GPT-5.4	204	126	77	1	61.8	99.5
SWE-bench Live	Sonnet 4.6	83	77	6	0	92.8	100.0

Takeaways. Majority ($\geq 2/3$) agreement is **97–100 %** across every condition for both blame and outcome, meaning the modal vote is almost always stable and reliable as a signal. Full unanimous agreement is somewhat lower — **62–85 %** for blame and **62–93 %** for outcome — which is expected: the taxonomy contains adjacent categories (*model_capability* vs. *agent_scaffolding*) that can legitimately both apply to the same failure, and the outcome three-way split (pass / almost / fail) places hard boundaries on a continuous quality gradient.

The hardest condition is SWE-bench Verified outcome for GPT-5.4, where unanimous agreement falls to 61.8 %. We attribute this to the high overall pass rate on that benchmark ($\approx 58\%$): many GPT-5.4 episodes produce partial fixes that sit ambiguously between *almost* and *fail*, a genuinely difficult judgement call even for human annotators. Notably, the *all-differ* rate (complete three-way disagreement) stays below 2.5 % in every condition, confirming that true confusion is rare and that majority voting is a robust aggregation strategy.

F Judge case studies

Table 5 shows a representative case in which all three judges unanimously converge, illustrating both the evidence-grounding mechanism and the blame attribution in practice. The episode is drawn from the SWE-bench Verified run (§6); the judge had access to the full step-indexed trajectory and the privileged evaluation output.

Table 5: Three independent judge outputs for episode `astropy__astropy-14365` (SWE-bench Verified, outcome *almost*). All three judges unanimously agree on primary blame and intervention; high-confidence evidence quotes pinpoint the unfixed downstream `if v == "NO"`: comparison that the agent read but failed to connect to its own regex fix. Evidence quotes are verbatim from the judge output; long code blocks are truncated at [...] to fit the column.

Field	Judge 1	Judge 2	Judge 3
Outcome	<i>almost</i>	<i>almost</i>	<i>almost</i>
Primary blame	<code>model_capability</code> (conf. 4/5)	<code>model_capability</code> (conf. 3/5)	<code>model_capability</code> (conf. 4/5)
Secondary	<code>agent_scaffolding</code>	–	<code>agent_scaffolding</code>
Summary	Added <code>re.IGNORECASE</code> to QDP command regex but missed downstream <code>if v == "NO"</code> : null-marker check; verified only syntax compilation, never ran the test suite.	Fix incomplete: <code>re.IGNORECASE</code> applied to command recognition; null-value sentinel "NO" still compared case-sensitively; crashes on lowercase "no" with <code>ValueError</code> .	Regex fix correct for command recognition; identical case-sensitivity bug in data-value processing loop not connected; <code>final_step</code> called without running any tests.
Key evidence	<pre> step 8 (verified): for v in line.split(delimiter): if v == "NO": ... else: [...] step 16 (verified): ValueError: could not convert string to float: 'no' </pre>	<pre> step 56 (verified, match 0.88): ValueError: could not convert string to float: 'no' astropy/io/ascii/qdp.py:317: ValueError </pre>	<pre> step 8 (verified): for v in line.split(delimiter): if v == "NO": ... else: [...] step 15 (verified): ACTION final_step: {} </pre>
Hypothesis	Change <code>"NO"</code> to <code>v.upper() == "NO"</code> throughout; mandate <code>pytest</code> run on the failing test before submit.	Run the actual failing test before submitting; would reveal the secondary failure immediately.	Add mandatory post-fix validation step (run failing reproduction case) to the agent workflow.
Intervention	<code>prompt_change</code> (conf. 3/5)	<code>prompt_change</code> (conf. 3/5)	<code>prompt_change</code> (conf. 4/5)

Table 6: Three independent judge outputs for episode `conan-io__conan-18028` (SWE-bench Live, outcome *failure*). The agent located `graph_builder.py` and the `skip_test` check at step 82, then immediately fell into a deterministic loop, re-issuing the same `grep` command 53 consecutive times and hitting `MAX_STEPS_REACHED` without writing a single line of code. All three judges agree on `scaffold_change` as the intervention; two of three assign primary blame to `agent_scaffolding`. Evidence quotes are verbatim from the judge output; long action strings are truncated at [...] to fit the column.

Field	Judge 1	Judge 2	Judge 3
Outcome	<i>failure</i>	<i>failure</i>	<i>failure</i>
Primary blame	<code>agent_scaffolding</code> (conf. 5/5)	<code>model_capability</code> (conf. 4/5)	<code>agent_scaffolding</code> (conf. 4/5)
Secondary	<code>model_capability</code>	<code>agent_scaffolding</code>	<code>model_capability</code>
Summary	Spent all 150 steps in read-only exploration, running identical <code>grep</code> commands dozens of times without writing a single code edit or calling <code>final_step</code> .	Spent all 150 steps in <code>grep/cat</code> exploration; from step 80 fell into a tight loop re-running the same <code>grep</code> with identical output each time.	Located <code>tools.graph:skip_test</code> in <code>graph_builder.py</code> at step 81, then immediately fell into a deterministic loop: same <code>grep</code> command 53 consecutive times, 0 code edits, <code>MAX_STEPS_REACHED</code> .
Key evidence	<i>steps 99, 101, 103</i> (verified, match 0.86): ACTION bash: <code>{...grep -rn "skip_test tools.graph:skip. .../graph_builder.py}</code> <i>steps 295, 297, 299</i> (verified, match 0.90): ACTION bash: <code>{...grep -rn "nodetest\b" conans/conan/ -include="*.py" grep -v "test/"}</code>	<i>step 81</i> (verified, match 0.86): ACTION bash: <code>{...grep -rn "skip_test tools.graph:skip. .../graph_builder.py}</code> <i>step 121</i> (verified, match 0.86): ACTION bash: <code>{...grep -rn "skip_test tools.graph:skip. .../graph_builder.py}</code>	<i>step 82</i> (verified): 216: <code>skip_test = node.conanfile.conf.get("tools.graph:skip_test",...)</code> 222: <code>if skip_test and require.test:</code> <i>step 83 → 151</i> (loop begins): ACTION bash: <code>{...grep -rn "skip_test tools.graph:skip_test" .../graph_builder.py}</code> <i>metadata</i> (step 0): "status": "MAX_STEPS_REACHED", "prompt_tokens": 6430181, "cached_tokens": 6267737
Hypothesis	Adding loop detection (block on N consecutive identical tool calls) and context compaction at a reasonable threshold would prevent this class of failure.	A scaffold with loop detection would have interrupted the cycle and forced a different action; the agent had located the relevant code but lacked the push to synthesize an edit.	Full history, temperature 0, and no compaction create a deterministic feedback loop once repetition starts; aborting after K consecutive identical actions would break the cycle.
Intervention	<code>scaffold_change</code> (conf. 5/5)	<code>scaffold_change</code> (conf. 3/5)	<code>scaffold_change</code> (conf. 5/5)

Table 7: Three independent judges on three WAA episodes (ep87, ep88, ep151; all VLC-domain, all scored 0.0), working on disjoint trajectories with no shared context. All three converge on the same eval-side root cause: the `vlcrc` path string passed to `execute_python_command` is re-parsed as Python source, silently collapsing `\v` to `U+000B` (vertical tab) and corrupting the path before `get_file()` sees it. A one-line `os.path.join` fix on the VM side flipped all five affected episodes from 0.0 to 1.0 on a single resumed run.

Field	Judge / ep87	Judge / ep88	Judge / ep151
Outcome	<i>should_have_been_rewarded</i>	<i>should_have_been_rewarded</i>	<i>should_have_been_rewarded</i>
Primary blame	<code>eval_brittle</code> (conf. 3/5)	<code>eval_brittle</code> (conf. 3/5)	<code>tool_failure</code> (conf. 2/5)
Task	Set VLC recording folder to Downloads	Set VLC recording folder to Desktop	Disable “Resize interface to video size” in VLC
Summary	Agent completed navigation (Preferences → [...] → Downloads → Save). Evaluator’s <code>get_vlc_config</code> path literal re-parsed as Python; <code>\v</code> → <code>U+000B</code> corrupted <code>\vlcrc</code> to <code>\u000blcrc</code> ; <code>get_file 500</code> , false <code>FileNotFoundError</code> .	Agent navigated Preferences and set Desktop as recording folder and saved. Evaluator’s <code>get_vlc_config</code> path literal re-parsed as Python; <code>\v</code> → <code>U+000B</code> corrupted <code>\vlcrc</code> to <code>\u000blcrc</code> ; <code>get_file 500</code> , score 0.0.	Agent navigated Preferences and unchecked ‘Resize interface to video size.’ Evaluator’s <code>get_vlc_config</code> path literal re-parsed as Python; <code>\v</code> → <code>U+000B</code> corrupted <code>\vlcrc</code> to <code>\u000blcrc</code> ; <code>get_file 500</code> , false <code>FileNotFoundError</code> .
Key evidence	<i>step 9</i> (verified): [ERROR] <code>get_file()</code> -- C:\Users\ Docker\AppData\Roaming\u000blc\u000blcrc	<i>step 9</i> (verified): [ERROR] <code>get_file()</code> -- C:\Users\ Docker\AppData\Roaming\u000blc\u000blcrc	<i>step 8</i> (verified): ACTION <code>run_pyautogui:</code> {"code": "pyautogui.click(784, 689) # Save"} <i>step 9</i> (verified): [ERROR] <code>get_file()</code> -- Roaming\u000blc\u000blcrc
Hypothesis	Path literal <code>\vlcrc</code> in <code>execute_python_command</code> re-parsed as Python source; <code>\v</code> → <code>U+000B</code> . Replace with <code>os.path.join</code> to eliminate escape-sequence literals.	Classic escape-sequence trap on the Python round-trip. Build path from <code>os.path.join</code> components so no <code>\v</code> literal survives re-parse.	<code>get_vlc_config</code> sends a Python expression to the VM; <code>\v</code> in <code>\vlcrc</code> collapses to <code>U+000B</code> silently. Switch to <code>os.path.join('~', ..., 'vlcrc')</code> .
Intervention	<code>eval_fix</code> (conf. 4/5)	<code>eval_fix</code> (conf. 4/5)	<code>eval_fix</code> (conf. 3/5)
Post-fix outcome	0.0 → 1.0 on resume (<code>is_vlc_recordings_folder</code> score 1.0)	0.0 → 1.0 on resume (<code>is_vlc_recordings_folder</code> score 1.0)	0.0 → 1.0 on resume (<code>is_vlc_resize_video_disabled</code> score 1.0)

Table 8: Three independent judge outputs for episode 716a6079_ep235 (**GPT-5.4 run**, OSWorld, outcome consensus: *failure*). Task: find `secret.docx` on the VM and copy its full path to the clipboard. File is at `/home/user/Data3/List3/secret.docx`. The three judges *disagree* on root cause: Judge 1 blames the model (agent used Ctrl+L in Nautilus, which copies the containing *directory* path, not the full file path, and rates the outcome *almost* — a near-miss sub-category of failure); Judges 2–3 blame infrastructure (`xsel` is not installed on the VM, so the evaluator `is_in_vm_clickboard` always reads an empty clipboard regardless of agent behavior). This split illustrates a compound failure: a model-capability weakness *and* a broken infrastructure-level evaluator co-occur, making the true verdict ambiguous. Compare with Table 9 (Sonnet run, same task): Sonnet used `xclip` correctly and received a unanimous *should_have_been_rewarded*.

Field	Judge 1	Judge 2	Judge 3
Outcome	<i>almost</i>	<i>failure</i>	<i>failure</i>
Primary blame	model_capability (conf. 3/5)	eval_brittle (conf. 4/5)	env_failure (conf. 4/5)
Secondary	action_space_limited	model_capability	model_capability
Summary	Agent found correct file via Nautilus search but copied only the containing <i>directory</i> path (Ctrl+L+Ctrl+C in Nautilus address bar) rather than the full file path including <code>/secret.docx</code> . Evaluator expected <code>/home/user/Data3/List3/secret.docx</code> but agent produced only <code>/home/user/Data3/List3</code> .	Evaluator depends on <code>xsel</code> to read VM clipboard, but <code>xsel</code> is not installed (returncode 127, <code>/bin/sh: 1: xsel: not found</code>). This makes scoring impossible for <i>any</i> agent. Separately, <code>agent</code> 's <code>ctrl+l+ctrl+c</code> method would produce directory path, not file path.	VM missing <code>xsel</code> binary that <code>is_in_vm_clickboard</code> relies on; evaluation command returned <code>/bin/sh: 1: xsel: not found</code> and empty output, making reward=0 inevitable regardless of agent behavior.
Key evidence	<i>step 29</i> (verified, match 0.94): ACTION hotkey: {"keys": "ctrl+l"} <i>step 31</i> (verified, match 0.94): ACTION hotkey: {"keys": "ctrl+c"} task config: "expected": "/home/user/Data3/List3/secret.docx"	<i>evaluator log</i> (match 0.34): {'error': '/bin/sh: 1: xsel: not found\n', 'returncode': 127, 'output': ''} <i>evaluator log</i> (match 0.52): reward=0.000000, evaluator=is_in_vm_clickboard <i>step 29</i> (verified, match 0.94): ACTION hotkey: {"keys": "ctrl+l"}	<i>evaluator log</i>: {'error': '/bin/sh: 1: xsel: not found\n', 'returncode': 127, 'output': ''} <i>evaluator</i>: is_in_vm_clickboard depends on xsel -clipboard -output
Hypothesis	Instruct agent to use file Properties dialog or right-click 'Copy Location' to obtain full file path; Ctrl+L in Nautilus yields only the directory.	Install <code>xsel</code> on VM base image, or update evaluator to fall back to <code>xclip -o -selection clipboard</code> ; also fix agent clipboard strategy.	VM image (cube-ubuntu-22-04 v1.0.0) does not include <code>xsel</code> ; install <code>xsel</code> or switch evaluator to <code>xclip -o</code> or <code>wl-paste</code> to fix all clipboard-check tasks.
Intervention	prompt_change (conf. 3/5)	eval_fix (conf. 5/5)	infra_fix (conf. 5/5)

Table 9: Three independent judge outputs for episode 716a6079_ep235 (**Sonnet run**, OS-World, outcome consensus: *should_have_been_rewarded*). *Same task as Table 8*. Unlike GPT-5.4 (which used Ctrl+L in Nautilus and copied only the directory path), the Sonnet agent correctly used `xclip -selection clipboard` in a terminal to copy the full file path `/home/user/Data3/List3/secret.docx`. All three judges agree: the correct agent action was irrelevant because the evaluator’s clipboard-reading command (`xsel -clipboard -output`) is not installed on the VM, so the clipboard always reads as empty. This cross-model pair is direct evidence that the same broken evaluator can produce a *model-capability failure* verdict for a weaker agent and a *should_have_been_rewarded* verdict for a stronger one — masking genuine capability differences behind infrastructure noise.

Field	Judge 1	Judge 2	Judge 3
Outcome	<i>should_have_been_rewarded</i>	<i>should_have_been_rewarded</i>	<i>should_have_been_rewarded</i>
Primary blame	eval_brittle (conf. 4/5)	eval_brittle (conf. 4/5)	eval_brittle (conf. 4/5)
Secondary	—	model_capability	env_failure
Summary	Agent correctly found the file at <code>/home/user/Data3/List3/secret.docx</code> and used <code>xclip</code> to copy its path to the clipboard, but evaluator’s clipboard-reading command (<code>xsel</code>) was not installed on the VM (returncode 127), causing it to always see an empty clipboard and return reward=0 regardless of agent actions.	Agent correctly identified and copied file path using <code>xclip -selection clipboard</code> , then called <code>done()</code> . Evaluator failed with <code>/bin/sh: 1: xsel: not found</code> (returncode 127) because <code>xsel</code> is not installed on the VM.	Agent correctly found <code>secret.docx</code> and copied its path to clipboard using <code>xclip</code> . Evaluator uses <code>xsel -clipboard -output</code> to verify; <code>xsel</code> is not installed (returncode 127), so comparison always sees empty string.
Key evidence	<i>evaluator log</i> (match 0.31): {'error': '/bin/sh: 1: xsel: not found\n', 'returncode': 127, 'output': ''} <i>agent reasoning, step 9</i> : “The path <code>/home/user/Data3/List3/secret.docx</code> has been copied to the clipboard using <code>xclip -selection clipboard</code> .”	<i>agent reasoning, step 7</i> : “The file has been found at <code>/home/user/Data3/List3/secret.docx</code> ” <i>evaluator log</i> (match 0.31): {'error': '/bin/sh: 1: xsel: not found\n', 'returncode': 127}	<i>step 15</i> (match 0.92): ACTION typing: {"text": "echo -n \"/home/user/Data3/List3/secret.docx\" xclip -selection clipboard"} <i>evaluator log</i> : <code>xsel: not found,</code> <code>returncode 127</code>
Hypothesis	VM image does not have <code>xsel</code> installed; install <code>xsel</code> or update evaluator to use <code>xdotool getclipboard</code> or <code>xclip -o</code> .	Install <code>xsel</code> on VM base image, or update evaluator to fall back to <code>xclip -o -selection clipboard</code> .	VM has <code>xclip</code> (installed) but not <code>xsel</code> ; evaluator’s getter always fails; fix evaluator to use <code>xclip -o -selection clipboard</code> .
Intervention	infra_fix (conf. 5/5)	infra_fix (conf. 5/5)	eval_fix (conf. 5/5)